

Chapter 38 — Knapsack Dynamic Programming

0. What You Will Learn in This Chapter

Knapsack DP is one of the most important DP families.

It appears whenever you must choose items under a constraint.

The core idea is:

For each item, decide whether to take it or skip it.

Knapsack problems usually involve:

- weights
- values
- capacity
- target sum
- subset sum
- partitioning
- choosing items once
- choosing items multiple times
- counting ways to form a target
- minimizing or maximizing value under a limit

By the end of this chapter, you should understand:

- What knapsack DP is
- 0/1 knapsack
- Unbounded knapsack
- Subset sum
- Equal partition
- Target sum transformation
- Counting combinations
- Counting permutations
- Boolean knapsack
- Min/max knapsack
- Loop-order rules
- Space optimization
- Common knapsack transformations
- How to identify hidden knapsack problems

Core problems covered in this chapter:

1. 0/1 Knapsack Classic
2. Unbounded Knapsack Classic
3. Partition Equal Subset Sum
4. Last Stone Weight II
5. Target Sum
6. Coin Change
7. Coin Change II
8. Combination Sum IV
9. Ones and Zeroes
10. Profitable Schemes
11. Shopping Offers
12. Minimum Cost For Tickets, knapsack view

13. Perfect Squares, unbounded view
 14. Integer Break, preview
 15. Form Largest Integer With Digits That Add Up to Target
-

1. What Is Knapsack DP?

Knapsack DP is a pattern where you choose items subject to a capacity or target.

Classic example:

You have items:

weight[i]
value[i]

You have a bag capacity:

W

Choose items to maximize total value without exceeding capacity.

Each item may be:

- used once
 - used multiple times
 - not used
-

2. Why It Is Called Knapsack

A knapsack is a bag.

You want to put items into the bag.

Each item has:

weight
value

The bag has limited capacity.

Question:

What is the best value I can carry?

This structure appears in many hidden forms:

- choose numbers to reach sum
 - choose coins to make amount
 - choose subset with target
 - split array into equal parts
 - choose strings under zero/one constraints
 - choose crimes under people/profit constraints
-

3. Main Knapsack Types

0/1 Knapsack	each item once	capacity backward	Partition Equal Subset

Type	Item Use	Common Loop Direction	Example
Unbounded Knapsack	item unlimited times	capacity forward	Coin Change
Bounded Knapsack	limited copies	special handling	less common
Boolean Knapsack	possible or not	OR transition	Subset Sum
Counting Knapsack	number of ways	add ways	Coin Change II
Min/Max Knapsack	optimize value	min/max transition	Coin Change

4. 0/1 Knapsack

In 0/1 knapsack, each item can be used at most once.

For each item:

take it
or skip it

Classic state:

$dp[i][w]$ = max value using first i items with capacity w

Transition:

skip = $dp[i - 1][w]$
take = $value[i - 1] + dp[i - 1][w - weight[i - 1]]$
 $dp[i][w] = \max(skip, take)$

5. 0/1 Knapsack Space Optimization

2D DP can be optimized to 1D.

For each item, loop capacity backward:

```
for (int item = 0; item < n; item++) {
    for (int w = capacity; w >= weight[item]; w--) {
        dp[w] = max(dp[w], dp[w - weight[item]] + value[item]);
    }
}
```

Why backward?

Because each item can be used only once.

Backward loop prevents using the same item multiple times in one iteration.

6. Unbounded Knapsack

In unbounded knapsack, each item can be used unlimited times.

Example:

Coin Change.

For each coin, you may use it multiple times.

Loop capacity forward:

```
for (int coin : coins) {  
    for (int amount = coin; amount <= target; amount++) {  
        dp[amount] = min(dp[amount], dp[amount - coin] + 1);  
    }  
}
```

Forward loop allows reusing the same coin.

7. Loop Direction Rule

This is one of the most important knapsack rules.

7.1 0/1 Knapsack

Each item used once.

Loop target backward:

```
for (int s = target; s >= x; s--) {  
    dp[s] = dp[s] || dp[s - x];  
}
```

7.2 Unbounded Knapsack

Item can be reused.

Loop target forward:

```
for (int s = x; s <= target; s++) {  
    dp[s] = dp[s] || dp[s - x];  
}
```

7.3 Counting Combinations

Usually loop items outside, target inside.

```
for (int coin : coins) {  
    for (int a = coin; a <= amount; a++) {  
        dp[a] += dp[a - coin];  
    }  
}
```

7.4 Counting Permutations

Usually loop target outside, items inside.

```
for (int a = 1; a <= target; a++) {  
    for (int num : nums) {  
        dp[a] += dp[a - num];  
    }  
}
```

8. When to Use Knapsack DP

Use knapsack DP when the problem asks:

8.1 Choose Subset With Target Sum

Examples:

- Partition Equal Subset Sum
- Last Stone Weight II

8.2 Min Coins / Ways to Make Amount

Examples:

- Coin Change
- Coin Change II

8.3 Assign Signs to Reach Target

Example:

- Target Sum

8.4 Choose Items Under Capacity

Examples:

- Ones and Zeroes
- Profitable Schemes

8.5 Maximize or Minimize Value Under Constraint

Examples:

- 0/1 Knapsack
 - Form Largest Integer With Digits That Add Up to Target
-

9. When NOT to Use Knapsack DP

9.1 No Capacity or Target

If there is no sum/weight/capacity constraint, knapsack may not apply.

9.2 Need Ordered Sequence

Use sequence DP.

Example:

LIS.

9.3 Need Contiguous Subarray

Use sliding window, prefix sum, Kadane, or interval DP.

9.4 Need Graph Connectivity

Use graph algorithms.

9.5 Need Generate All Combinations

Use backtracking.

10. Mental Models

10.1 Backpack Capacity

Each item consumes capacity.

You decide whether it fits.

10.2 Target Filling

You are trying to fill exactly a target sum.

Can I make sum s ?

How many ways can I make s ?

What is min cost to make s ?

10.3 Take or Skip

For 0/1 knapsack:

do not take item

take item once

10.4 Reuse Allowed

For unbounded knapsack:

after taking item, item remains available

11. Pattern Recognition Checklist

Ask:

16. Is there a target sum or capacity?
 17. Are there items/numbers/coins?
 18. Do I choose some items?
 19. Can each item be used once or multiple times?
 20. Is the objective possible / count / min / max?
 21. Does order matter?
 22. Is this subset sum?
 23. Is this coin change?
 24. Is there a hidden transformation to target sum?
 25. Does loop direction matter?
-

12. C++ Tools Needed

12.1 Boolean DP

```
vector<bool> dp(target + 1, false);  
dp[0] = true;
```

12.2 Min DP

```
const int INF = 1e9;  
vector<int> dp(amount + 1, INF);  
dp[0] = 0;
```

12.3 Count DP

```
vector<int> dp(amount + 1, 0);  
dp[0] = 1;
```

12.4 Max DP

```
vector<int> dp(capacity + 1, 0);
```

13. Core Templates

Template 1 — 0/1 Boolean Knapsack

```
vector<bool> dp(target + 1, false);  
dp[0] = true;  
  
for (int x : nums) {  
    for (int s = target; s >= x; s--) {  
        dp[s] = dp[s] || dp[s - x];  
    }  
}
```

Template 2 — 0/1 Max Value Knapsack

```
vector<int> dp(capacity + 1, 0);  
  
for (int i = 0; i < n; i++) {  
    for (int w = capacity; w >= weight[i]; w--) {  
        dp[w] = max(dp[w], dp[w - weight[i]] + value[i]);  
    }  
}
```

Template 3 — Unbounded Min Coins

```
const int INF = 1e9;  
vector<int> dp(amount + 1, INF);  
  
dp[0] = 0;  
  
for (int a = 1; a <= amount; a++) {  
    for (int coin : coins) {  
        if (a >= coin && dp[a - coin] != INF) {  
            dp[a] = min(dp[a], dp[a - coin] + 1);  
        }  
    }  
}
```

```
}
}
```

Template 4 — Count Combinations

```
vector<int> dp(amount + 1, 0);
dp[0] = 1;

for (int coin : coins) {
    for (int a = coin; a <= amount; a++) {
        dp[a] += dp[a - coin];
    }
}
```

Template 5 — Count Permutations

```
vector<int> dp(target + 1, 0);
dp[0] = 1;

for (int a = 1; a <= target; a++) {
    for (int x : nums) {
        if (a >= x) {
            dp[a] += dp[a - x];
        }
    }
}
```

14. Major Knapsack Variations

0/1 possible	dp[sum] boolean	Partition Equal Subset
0/1 max value	dp[capacity] max	Classic Knapsack
Unbounded min	dp[amount] min	Coin Change
Unbounded count combinations	dp[amount] ways	Coin Change II
Count permutations	dp[target] ordered ways	Combination Sum IV
Multi-constraint	dp[zeros][ones]	Ones and Zeroes

Variation	State	Example
Target transformation	subset sum	Target Sum
3D knapsack	index/capacity/profit	Profitable Schemes

15. Full Problem Walkthrough 1 — 0/1 Knapsack Classic

15.1 Problem Summary

Given items with weights and values.
Choose items with total weight at most capacity.
Maximize total value.
Each item can be used once.

15.2 State

$dp[w]$ = maximum value achievable with capacity w

15.3 Transition

For each item:

$dp[w] = \max(dp[w], dp[w - \text{weight}] + \text{value})$

Loop w backward.

15.4 C++ Code

```
int knapsack01(vector<int>& weights, vector<int>& values, int capacity) {
    int n = weights.size();

    vector<int> dp(capacity + 1, 0);

    for (int i = 0; i < n; i++) {
        for (int w = capacity; w >= weights[i]; w--) {
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }

    return dp[capacity];
}
```

15.5 Complexity

Time:

$O(n * \text{capacity})$

Space:

$O(\text{capacity})$

16. Full Problem Walkthrough 2 — Unbounded Knapsack Classic

16.1 Problem Summary

Same as knapsack, but each item can be used unlimited times.

16.2 State

$dp[w]$ = maximum value achievable with capacity w

16.3 Transition

Loop capacity forward:

```
for (int w = weights[i]; w <= capacity; w++)
```

16.4 C++ Code

```
int unboundedKnapsack(vector<int>& weights, vector<int>& values, int capacity)
{
    int n = weights.size();

    vector<int> dp(capacity + 1, 0);

    for (int i = 0; i < n; i++) {
        for (int w = weights[i]; w <= capacity; w++) {
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }

    return dp[capacity];
}
```

16.5 Complexity

Time:

$O(n * \text{capacity})$

Space:

$O(\text{capacity})$

17. Full Problem Walkthrough 3 — LeetCode 416: Partition Equal Subset Sum

17.1 Problem Summary

Given nums, return true if array can be partitioned into two subsets with equal sum.

17.2 How to Identify Knapsack

Need subset with sum:

total / 2

Each number used once.

This is 0/1 boolean knapsack.

17.3 C++ Code

```
class Solution {
public:
    bool canPartition(vector<int>& nums) {
        int total = accumulate(nums.begin(), nums.end(), 0);

        if (total % 2 == 1) {
            return false;
        }

        int target = total / 2;

        vector<bool> dp(target + 1, false);
        dp[0] = true;

        for (int x : nums) {
            for (int s = target; s >= x; s--) {
                dp[s] = dp[s] || dp[s - x];
            }
        }

        return dp[target];
    }
};
```

17.4 Why Loop Backward?

Each number can be used once.

Backward loop prevents using the same number multiple times.

17.5 Complexity

Time:

$O(n * \text{target})$

Space:

$O(\text{target})$

18. Full Problem Walkthrough 4 — LeetCode 1049: Last Stone Weight II

18.1 Problem Summary

Smash stones to minimize final remaining weight.

Equivalent to partition stones into two groups with sums as close as possible.

Answer:

```
total - 2 * bestSubsetSum
```

where $\text{bestSubsetSum} \leq \text{total} / 2$.

18.2 How to Identify Knapsack

Choose subset sum closest to half.

Each stone used once.

0/1 knapsack.

18.3 C++ Code

```
class Solution {
public:
    int lastStoneWeightII(vector<int>& stones) {
        int total = accumulate(stones.begin(), stones.end(), 0);
        int target = total / 2;

        vector<bool> dp(target + 1, false);
        dp[0] = true;

        for (int stone : stones) {
            for (int s = target; s >= stone; s--) {
                dp[s] = dp[s] || dp[s - stone];
            }
        }

        for (int s = target; s >= 0; s--) {
            if (dp[s]) {
                return total - 2 * s;
            }
        }

        return 0;
    }
};
```

18.4 Complexity

Time:

$O(n * \text{target})$

Space:

$O(\text{target})$

19. Full Problem Walkthrough 5 — LeetCode 494: Target Sum

19.1 Problem Summary

Assign + or - signs to each number so expression equals target.
Return number of ways.

19.2 Transformation

Let:

P = sum of positive group
 N = sum of negative group

We need:

$P - N = \text{target}$
 $P + N = \text{total}$

Add:

$2P = \text{target} + \text{total}$
 $P = (\text{target} + \text{total}) / 2$

So problem becomes:

count subsets with sum P

19.3 C++ Code

```
class Solution {
public:
    int findTargetSumWays(vector<int>& nums, int target) {
        int total = accumulate(nums.begin(), nums.end(), 0);

        if (abs(target) > total) {
            return 0;
        }

        if ((target + total) % 2 != 0) {
            return 0;
        }
    }
};
```

```

int subsetTarget = (target + total) / 2;

vector<int> dp(subsetTarget + 1, 0);
dp[0] = 1;

for (int x : nums) {
    for (int s = subsetTarget; s >= x; s--) {
        dp[s] += dp[s - x];
    }
}

return dp[subsetTarget];
}
};

```

19.4 Complexity

Time:

$O(n * \text{subsetTarget})$

Space:

$O(\text{subsetTarget})$

19.5 Common Mistake

Forgetting parity check:

target + total must be even

20. Full Problem Walkthrough 6 — LeetCode 322: Coin Change

20.1 Problem Summary

Return minimum number of coins needed to make amount.

Coins can be used unlimited times.

20.2 How to Identify Knapsack

This is unbounded minimum knapsack.

20.3 C++ Code

```

class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        const int INF = 1e9;

        vector<int> dp(amount + 1, INF);
    }
};

```

```

dp[0] = 0;

for (int a = 1; a <= amount; a++) {
    for (int coin : coins) {
        if (a >= coin && dp[a - coin] != INF) {
            dp[a] = min(dp[a], dp[a - coin] + 1);
        }
    }
}

return dp[amount] == INF ? -1 : dp[amount];
}
};

```

20.4 Complexity

Time:

$O(\text{amount} * \text{coins})$

Space:

$O(\text{amount})$

21. Full Problem Walkthrough 7 — LeetCode 518: Coin Change II

21.1 Problem Summary

Return number of combinations that make amount.

Coins can be used unlimited times.

Order does not matter.

21.2 How to Identify Knapsack

Unbounded counting combinations.

Loop coins outside.

21.3 C++ Code

```

class Solution {
public:
    int change(int amount, vector<int>& coins) {
        vector<int> dp(amount + 1, 0);
        dp[0] = 1;

        for (int coin : coins) {
            for (int a = coin; a <= amount; a++) {
                dp[a] += dp[a - coin];
            }
        }
    }
};

```

```
    }  
  
    return dp[amount];  
}  
};
```

21.4 Why Coins Outside?

This avoids counting different orders as separate ways.

Example:

```
1 + 2  
2 + 1
```

should count once.

21.5 Complexity

Time:

$O(\text{amount} * \text{coins})$

Space:

$O(\text{amount})$

22. Full Problem Walkthrough 8 — LeetCode 377: Combination Sum IV

22.1 Problem Summary

Given nums and target.

Return number of possible combinations where order matters.

Example:

```
nums = [1,2,3]  
target = 4
```

Different orders count separately.

22.2 How to Identify Knapsack

This is counting permutations.

Loop target outside, nums inside.

22.3 C++ Code

```
class Solution {  
public:  
    int combinationSum4(vector<int>& nums, int target) {  
        vector<unsigned int> dp(target + 1, 0);  
        dp[0] = 1;  
  
        for (int sum = 1; sum <= target; sum++) {  
            for (int x : nums) {
```

```

        if (sum >= x) {
            dp[sum] += dp[sum - x];
        }
    }
}

return dp[target];
}
};

```

22.4 Why Target Outside?

Because order matters.

For each target sum, we try every possible last number.

22.5 Complexity

Time:

$O(\text{target} * \text{nums.size}())$

Space:

$O(\text{target})$

23. Full Problem Walkthrough 9 — LeetCode 474: Ones and Zeroes

23.1 Problem Summary

Given binary strings and limits:

m zeros
n ones

Return maximum number of strings you can choose.

Each string can be used once.

23.2 How to Identify Knapsack

Each string is an item.

Weight has two dimensions:

zeros
ones

Value is:

1 string

This is 0/1 knapsack with two capacities.

23.3 State

$\text{dp}[z][o]$ = max strings using at most z zeros and o ones

23.4 C++ Code

```

class Solution {
public:
    pair<int, int> countZerosOnes(string& s) {
        int zeros = 0;
        int ones = 0;

        for (char c : s) {
            if (c == '0') {
                zeros++;
            } else {
                ones++;
            }
        }

        return {zeros, ones};
    }

    int findMaxForm(vector<string>& strs, int m, int n) {
        vector<vector<int>>> dp(m + 1, vector<int>(n + 1, 0));

        for (string& s : strs) {
            auto [zeros, ones] = countZerosOnes(s);

            for (int z = m; z >= zeros; z--) {
                for (int o = n; o >= ones; o--) {
                    dp[z][o] = max(dp[z][o],
                                    dp[z - zeros][o - ones] + 1);
                }
            }
        }

        return dp[m][n];
    }
};

```

23.5 Why Loops Backward?

Each string can be used once.

23.6 Complexity

Time:

$O(\text{strs.size()} * m * n)$

Space:

$O(m * n)$

24. Full Problem Walkthrough 10 — LeetCode 879: Profitable Schemes

24.1 Problem Summary

There are crimes.

Each crime needs group members and gives profit.

Choose crimes with at most n members and at least minProfit profit.

Return number of schemes.

24.2 How to Identify Knapsack

Each crime can be used once.

Constraints:

people used
profit achieved

This is 0/1 counting knapsack with capped profit.

24.3 State

$\text{dp}[\text{people}][\text{profit}] = \text{number of schemes}$

Profit is capped at minProfit .

24.4 C++ Code

```
class Solution {
public:
    int profitableSchemes(int n, int minProfit,
                        vector<int>& group,
                        vector<int>& profit) {
        const int MOD = 1000000007;

        vector<vector<int>> dp(n + 1, vector<int>(minProfit + 1, 0));
        dp[0][0] = 1;

        for (int i = 0; i < group.size(); i++) {
            int members = group[i];
            int gain = profit[i];

            for (int people = n; people >= members; people--) {
                for (int p = minProfit; p >= 0; p--) {
                    int newProfit = min(minProfit, p + gain);

                    dp[people][newProfit] =
                        (dp[people][newProfit] + dp[people - members][p]) % MOD;
                }
            }
        }
    }
};
```

```

    }
}

int answer = 0;

for (int people = 0; people <= n; people++) {
    answer = (answer + dp[people][minProfit]) % MOD;
}

return answer;
}
};

```

24.5 Complexity

Time:

$O(\text{crimes} * n * \text{minProfit})$

Space:

$O(n * \text{minProfit})$

25. Full Problem Walkthrough 11 — Shopping Offers

25.1 Problem Summary

Given item prices, special bundle offers, and needs.
Return minimum cost to satisfy needs.

25.2 How to Identify DP

State is current remaining needs.
This is knapsack-like with memoization over vector state.

25.3 C++ Code

```

class Solution {
public:
    map<vector<int>, int> memo;

    int dfs(vector<int>& price,
            vector<vector<int>>& special,
            vector<int> needs) {
        if (memo.count(needs)) {
            return memo[needs];
        }

        int best = 0;

```

```

for (int i = 0; i < needs.size(); i++) {
    best += needs[i] * price[i];
}

for (auto& offer : special) {
    vector<int> nextNeeds = needs;
    bool valid = true;

    for (int i = 0; i < needs.size(); i++) {
        if (offer[i] > nextNeeds[i]) {
            valid = false;
            break;
        }

        nextNeeds[i] -= offer[i];
    }

    if (valid) {
        best = min(best,
            offer.back() + dfs(price, special, nextNeeds));
    }
}

memo[needs] = best;
return best;
}

int shoppingOffers(vector<int>& price,
    vector<vector<int>>& special,
    vector<int>& needs) {
    return dfs(price, special, needs);
}
};

```

25.4 Complexity

Depends on number of possible need states.

This is memoized multidimensional knapsack.

26. Full Problem Walkthrough 12 — Minimum Cost For Tickets, Knapsack View

26.1 Problem Summary

Given travel days and pass costs for:

1-day, 7-day, 30-day

Return minimum cost to cover all travel days.

26.2 Knapsack View

At each day, choose one pass type.

Pass covers a range of future days.

This is 1D DP over calendar days.

26.3 C++ Code

```
class Solution {
public:
    int mincostTickets(vector<int>& days, vector<int>& costs) {
        unordered_set<int> travel(days.begin(), days.end());

        int lastDay = days.back();

        vector<int> dp(lastDay + 1, 0);

        for (int day = 1; day <= lastDay; day++) {
            if (!travel.count(day)) {
                dp[day] = dp[day - 1];
            } else {
                int one = dp[max(0, day - 1)] + costs[0];
                int seven = dp[max(0, day - 7)] + costs[1];
                int thirty = dp[max(0, day - 30)] + costs[2];

                dp[day] = min({one, seven, thirty});
            }
        }

        return dp[lastDay];
    }
};
```

27. Full Problem Walkthrough 13 — Perfect Squares, Unbounded View

Perfect Squares is unbounded knapsack.
Squares are reusable.

```
class Solution {
public:
    int numSquares(int n) {
        vector<int> dp(n + 1, 1e9);
        dp[0] = 0;

        for (int square = 1; square * square <= n; square++) {
            int value = square * square;

            for (int amount = value; amount <= n; amount++) {
                dp[amount] = min(dp[amount], dp[amount - value] + 1);
            }
        }

        return dp[n];
    }
};
```

28. Full Problem Walkthrough 14 — Integer Break, Preview

28.1 Problem Summary

Break integer n into at least two positive integers to maximize product.

28.2 DP State

$dp[i]$ = maximum product after breaking i

28.3 Transition

Try first part j :

$dp[i] = \max(dp[i], \max(j * (i - j), j * dp[i - j]))$

This is not classic knapsack but has similar split-choice thinking.

29. Full Problem Walkthrough 15 — LeetCode 1449: Form Largest

Integer With Digits That Add Up to Target

29.1 Problem Summary

Digits 1..9 have costs.

Build largest integer such that total cost equals target.

Return string.

29.2 How to Identify Knapsack

Each digit can be used multiple times.

Need maximize number length first, then lexicographic value.

Unbounded knapsack over target cost.

29.3 C++ Code

```
class Solution {
public:
    string largestNumber(vector<int>& cost, int target) {
        vector<int> dp(target + 1, -1e9);
        dp[0] = 0;

        for (int t = 1; t <= target; t++) {
            for (int d = 1; d <= 9; d++) {
                int c = cost[d - 1];

                if (t >= c) {
                    dp[t] = max(dp[t], dp[t - c] + 1);
                }
            }
        }

        if (dp[target] < 0) {
            return "0";
        }

        string result;
        int t = target;

        for (int d = 9; d >= 1; d--) {
            int c = cost[d - 1];

            while (t >= c && dp[t] == dp[t - c] + 1) {
                result.push_back('0' + d);
                t -= c;
            }
        }
    }
};
```

```
    }  
  }  
  
  return result;  
}  
};
```

29.4 Complexity

Time:

$O(9 * \text{target})$

Space:

$O(\text{target})$

30. Common Knapsack Edge Cases

30.1 Target 0

Usually:

```
dp[0] = true  
dp[0] = 0  
dp[0] = 1
```

depending on problem.

30.2 Impossible Target

Return false, -1, or 0 ways.

30.3 Odd Total Sum

Partition Equal Subset Sum impossible.

30.4 Zeros in Input

Target Sum with zeros can multiply ways.

30.5 Large Counts

Use modulo or long long.

30.6 Loop Direction

Most common bug.

Backward for 0/1.

Forward for unbounded.

31. Common Knapsack Mistakes

Mistake 1 — Wrong Loop Direction

0/1 knapsack must loop backward.

Unbounded knapsack often loops forward.

Mistake 2 — Counting Permutations Instead of Combinations

Coin Change II needs coins outside loop.
Combination Sum IV needs target outside loop.

Mistake 3 — Forgetting Transformation

Target Sum transforms into subset sum.
Last Stone Weight II transforms into closest half subset.

Mistake 4 — Bad Initialization

For min DP:

```
dp[0] = 0  
others = INF
```

For count DP:

```
dp[0] = 1
```

For boolean DP:

```
dp[0] = true
```

Mistake 5 — Overflow in Count

Use modulo or wider type when needed.

32. Interview Explanation Templates

32.1 0/1 Knapsack

Each item can be either taken or skipped. I use DP over capacity, and loop capacity backward so the same item is not used more than once.

32.2 Unbounded Knapsack

Each item can be reused. I loop capacity forward so the current item can contribute multiple times.

32.3 Partition Equal Subset

If total sum is even, I need to know whether any subset can make half of total. That is a 0/1 subset-sum DP.

32.4 Coin Change

I define $dp[a]$ as the minimum coins needed for amount a . For each amount, I try every coin as the last coin used.

32.5 Coin Change II

I count combinations, so I process coins one by one and update amounts forward. This avoids counting different orders separately.

33. Complexity Patterns

0/1 Knapsack	$O(nW)$	$O(W)$

Problem	Time	Space
Unbounded Knapsack	$O(nW)$	$O(W)$
Partition Equal Subset	$O(n * \text{sum})$	$O(\text{sum})$
Last Stone Weight II	$O(n * \text{sum})$	$O(\text{sum})$
Target Sum	$O(n * \text{subsetTarget})$	$O(\text{subsetTarget})$
Coin Change	$O(\text{amount} * \text{coins})$	$O(\text{amount})$
Coin Change II	$O(\text{amount} * \text{coins})$	$O(\text{amount})$
Combination Sum IV	$O(\text{target} * \text{nums})$	$O(\text{target})$
Ones and Zeroes	$O(\text{strs} * m * n)$	$O(mn)$
Profitable Schemes	$O(\text{crimes} * \text{people} * \text{profit})$	$O(\text{people} * \text{profit})$
Shopping Offers	exponential states with memo	memo size

34. Time-Boxed Practice Guidance

Intro Knapsack

Target:

30–45 minutes

Problems:

- Partition Equal Subset Sum
- Coin Change
- Coin Change II

Medium Knapsack

Target:

45–60 minutes

Problems:

- Target Sum
- Last Stone Weight II
- Combination Sum IV
- Ones and Zeroes

Hard Knapsack

Target:

60–90 minutes

Problems:

- Profitable Schemes
- Shopping Offers
- Form Largest Integer
- Number of Ways to Earn Points

35. Expanded Practice Set

35.1 Must-Do Medium

- 26. Partition Equal Subset Sum
- 27. Last Stone Weight II
- 28. Target Sum
- 29. Coin Change
- 30. Coin Change II
- 31. Combination Sum IV
- 32. Perfect Squares
- 33. Ones and Zeroes
- 34. Minimum Cost For Tickets

35.2 Strong Interview Practice

- 35. Profitable Schemes
- 36. Shopping Offers
- 37. Form Largest Integer With Digits That Add Up to Target
- 38. Number of Ways to Earn Points
- 39. Integer Break
- 40. Count Number of Ways to Place Houses
- 41. Tallest Billboard

35.3 Hard / Stretch

- 42. Tallest Billboard
 - 43. Profitable Schemes
 - 44. Shopping Offers
 - 45. Form Largest Integer With Digits That Add Up to Target
 - 46. Count All Possible Routes
 - 47. Minimum Cost to Cut a Stick, interval DP not knapsack
 - 48. Number of Music Playlists, advanced counting DP
-

36. Suggested Study Order

Step 1 — Boolean 0/1 Knapsack

Start with:

- 49. Partition Equal Subset Sum
- 50. Last Stone Weight II

Goal:

Learn subset-sum possibility.

Step 2 — Target Transformation

Solve:

- 51. Target Sum

Goal:

Learn algebraic DP transformation.

Step 3 — Unbounded Min DP

Solve:

- 52. Coin Change
- 53. Perfect Squares

Goal:

Learn min count to form amount.

Step 4 — Count Combinations vs Permutations

Solve:

- 54. Coin Change II
- 55. Combination Sum IV

Goal:

Master loop order.

Step 5 — Multi-Constraint Knapsack

Solve:

- 56. Ones and Zeroes
- 57. Profitable Schemes

Goal:

Learn multiple capacity dimensions.

Step 6 — Advanced Knapsack

Solve:

- 58. Shopping Offers
- 59. Form Largest Integer
- 60. Tallest Billboard

Goal:

Handle complex state and reconstruction.

37. Pattern Decision Table

subset sum possible	0/1 boolean	Partition Equal Subset
closest half sum	0/1 boolean/max	Last Stone Weight II
assign + / - signs	subset transform	Target Sum
min coins amount	unbounded min	Coin Change
count coin combinations	unbounded combination count	Coin Change II
ordered combinations	permutation count	Combination Sum IV
item has two costs	2D capacity	Ones and Zeroes
at least profit with people limit	2D count knapsack	Profitable Schemes
bundle offers	memoized multidimensional	Shopping Offers
build largest by cost	unbounded max length +	Largest Number

Problem Clue	Knapsack Type	Example
	reconstruction	

38. Mini Problem 1 — Number of Ways to Earn Points

Typical contest knapsack.
Each question type:

count, marks

Can use up to count copies.
This is bounded knapsack.
Basic approach:

for each type:
 for score from target down to 0:
 for use from 1 to count:
 update score + use * marks

This chapter focuses on 0/1 and unbounded. Bounded knapsack can be optimized later.

39. Mini Problem 2 — Tallest Billboard, Preview

State:

$dp[diff]$ = tallest shorter support height for this difference

For each rod:

- put on taller side
- put on shorter side
- skip

This is advanced knapsack over difference.

40. Mini Problem 3 — Count Ways With Modulo

For counting DP:

```
const int MOD = 1000000007;
```

```
dp[next] = (dp[next] + dp[current]) % MOD;
```

Use whenever answer may be large.

41. Mini Problem 4 — 2D 0/1 Knapsack Compression

For two capacities:

```
for item:
    for c1 from cap1 down to cost1:
        for c2 from cap2 down to cost2:
            dp[c1][c2] = max(dp[c1][c2],
                              dp[c1 - cost1][c2 - cost2] + value);
```

Used in Ones and Zeroes.

42. Common Edge Case Checklist

Before submitting knapsack code, test:

```
target = 0
empty items
single item
item larger than target
all items larger than target
duplicate coins
zeros in nums
odd total sum
target larger than total
amount impossible
multiple ways with same total
```

43. Debugging Knapsack DP

If knapsack DP fails:

61. Is each item used once or unlimited?
 62. Is loop direction correct?
 63. Is `dp[0]` initialized correctly?
 64. Are impossible states initialized correctly?
 65. Does order matter?
 66. Are you counting combinations or permutations?
 67. Did you transform target correctly?
 68. Are you using modulo when required?
 69. Did you handle zero values?
-

44. What to Master Before Chapter 39

Before moving to Interval DP, make sure you can:

- Explain 0/1 knapsack
 - Explain unbounded knapsack
 - Explain backward vs forward loop
 - Solve Partition Equal Subset Sum
 - Solve Last Stone Weight II
 - Solve Target Sum
 - Solve Coin Change
 - Solve Coin Change II
 - Solve Combination Sum IV
 - Solve Ones and Zeroes
 - Understand count vs min vs max DP
 - Initialize boolean/count/min DP correctly
 - Recognize hidden subset-sum transformations
-

45. Final Cheat Sheet

0/1 Boolean

```
dp[0] = true;

for (int x : nums) {
    for (int s = target; s >= x; s--) {
        dp[s] = dp[s] || dp[s - x];
    }
}
```

0/1 Max

```
for (int i = 0; i < n; i++) {
    for (int w = capacity; w >= weight[i]; w--) {
        dp[w] = max(dp[w], dp[w - weight[i]] + value[i]);
    }
}
```

Unbounded Min

```
dp[0] = 0;

for (int a = 1; a <= amount; a++) {
    for (int coin : coins) {
        dp[a] = min(dp[a], dp[a - coin] + 1);
    }
}
```

```
}  
}
```

Count Combinations

```
dp[0] = 1;  
  
for (int coin : coins) {  
    for (int a = coin; a <= amount; a++) {  
        dp[a] += dp[a - coin];  
    }  
}
```

Count Permutations

```
dp[0] = 1;  
  
for (int a = 1; a <= target; a++) {  
    for (int x : nums) {  
        if (a >= x) {  
            dp[a] += dp[a - x];  
        }  
    }  
}
```

46. Core Takeaway

Knapsack DP is about choosing items under constraints.

The most important questions are:

Can each item be used once or multiple times?

Am I checking possibility, counting ways, or optimizing value?

Does order matter?

What is the capacity or target?

The most important implementation rule is loop direction:

0/1 knapsack -> loop backward

unbounded knapsack -> loop forward

The most common hidden transformations are:

Partition Equal Subset -> subset sum total / 2

Target Sum -> subset sum (target + total) / 2

Last Stone Weight II -> closest subset sum to total / 2

If you master these transformations and loop rules, most knapsack problems become systematic.